# Práctica 1. Contenedor Podman persistente con MongoDB

Luis Daniel Morales Villarauz

22020790

Instituto Tecnológico de Veracruz

Base de Datos NoSQL

Jose Nahum Prieto Velazquez

26 de octubre de 2026

# Práctica 1. Contenedor Podman persistente con MongoDB

## Objetivo

Desplegar un servidor de MongoDB dentro de un contenedor administrado con Podman y comprobar que la información almacenada se conserva aun después de eliminar el contenedor, mediante el uso de un volumen con nombre.

## Introducción

Un contenedor empaqueta una aplicación junto con sus dependencias y se ejecuta de forma aislada del sistema anfitrión. Por diseño, el sistema de archivos de un contenedor es efímero: cuando el contenedor se elimina, todo lo que se escribió en él se pierde. Para un gestor de bases de datos esto es un problema, ya que los datos deben sobrevivir a reinicios, actualizaciones de imagen o recreaciones del servicio.

Podman es un motor de contenedores compatible con el formato OCI que no requiere un demonio en segundo plano y puede ejecutarse sin privilegios de administrador (modo *rootless*). Para lograr persistencia, Podman ofrece los volúmenes: directorios administrados por el motor que se montan dentro del contenedor y cuyo ciclo de vida es independiente del contenedor que los usa. En esta práctica se monta un volumen en `/data/db`, que es la ruta donde MongoDB guarda sus archivos de datos.

## Entorno de trabajo

La práctica se realizó en un equipo con CachyOS (Linux, kernel 7.2.9), Podman 6.1.4 en modo *rootless*, la imagen oficial `mongo:8` y el cliente `mongosh` instalado en el anfitrión.

# Desarrollo

## Paso 1. Verificación de Podman

Se comprobó la versión instalada y las características principales del entorno. El campo `Rootless: true` confirma que los contenedores se ejecutan con el usuario normal, sin permisos de superusuario.

**Figura 1**

*Versión e información del entorno de Podman*

```
$ podman --version
podman version 6.1.4

$ podman info --format 'Arquitectura: {{.Host.Arch}}
Distribucion: {{.Host.Distribution.Distribution}}
Kernel: {{.Host.Kernel}}
Cgroups: {{.Host.CgroupsVersion}}
Rootless: {{.Host.Security.Rootless}}
Driver de almacenamiento: {{.Store.GraphDriverName}}
Runtime OCI: {{.Host.OCIRuntime.Name}}'
Arquitectura: amd64
Distribucion: cachyos
Kernel: 7.2.9-2-cachyos
Cgroups: v2
Rootless: true
Driver de almacenamiento: overlay
Runtime OCI: runc
```

**Paso 2. Descarga de la imagen de MongoDB**

Se descargó la imagen oficial de MongoDB versión 8 desde Docker Hub. Se usa el nombre completo del registro (`docker.io/library/`) para que Podman no tenga que preguntar de qué registro obtenerla.

**Figura 2**

*Descarga de la imagen mongo:8*

```
$ podman pull docker.io/library/mongo:8
Trying to pull docker.io/library/mongo:8...
Getting image source signatures
Copying blob sha256:9d262346b5a5406d2a71fcba5b167060a17a4514c8e62338ed4cb1f1b6543d3a
Copying blob sha256:2a528370171490484547c3b3e49559b630c56d23a8cd67a158a44d28cc9f4f8b
Copying blob sha256:6abf326b33649389bb966e016adb23b9780579f28f94038edd3270ac7186cc84
Copying blob sha256:f2b4ef06ff01a67d4bed7657a4ae379a7d07934b7c3e743454a22a0d5275fb2b
Copying blob sha256:c344c51cc74fb72bef4bd09c4c09e874426c8dfa6848bb6d6d3a7ceb25630ab8
Copying blob sha256:035e14d0a929d7a956a16f2fd7750baed5a55595fef9008759b8aaa1e55f4c4d
Copying blob sha256:537537bbde4ccb6d8004dd9c1ca226750647875043a83481d96aad253d5e5e64
Copying blob sha256:d90d7952f291d5750cd9c7313a466185807aa40c25053bfb3567635bb377fa31
Copying config sha256:3aeccd8719807aad7f530d682186236bdafdbd48dda85a582b23cf551721d996
Writing manifest to image destination
3aeccd8719807aad7f530d682186236bdafdbd48dda85a582b23cf551721d996

$ podman images docker.io/library/mongo
REPOSITORY               TAG         IMAGE ID      CREATED     SIZE
docker.io/library/mongo  8           3aeccd871980  9 days ago  838 MB
```

### Paso 3. Creación del volumen persistente

Se creó un volumen con nombre, `mongo-data`. Al inspeccionarlo se observa su punto de montaje en el anfitrión, dentro del almacenamiento del usuario, y que `MountCount` es 0 porque todavía ningún contenedor lo utiliza.

**Figura 3**

*Creación e inspección del volumen mongo-data*

```
$ podman volume create mongo-data
mongo-data

$ podman volume ls --filter name=mongo-data
DRIVER      VOLUME NAME
local       mongo-data

$ podman volume inspect mongo-data
[
     {
          "Name": "mongo-data",
          "Driver": "local",
          "Mountpoint":
"/home/danielmorales/.local/share/containers/storage/volumes/mongo-data/_data",
          "CreatedAt": "2026-10-26T00:29:14.337158681-06:00",
          "Labels": {},
          "Scope": "local",
          "Options": {},
          "MountCount": 0,
          "NeedsCopyUp": true,
          "NeedsChown": true,
          "LockNumber": 0
     }
]
```

### Paso 4. Ejecución del contenedor con el volumen montado

Se levantó el contenedor `mongo-persistente` en segundo plano (`-d`), publicando el puerto 27017 en el anfitrión (`-p`) y montando el volumen en `/data/db` (`-v`). Las variables de entorno `MONGO_INITDB_ROOT_USERNAME` y `MONGO_INITDB_ROOT_PASSWORD` crean el usuario administrador la primera vez que se inicializa la base de datos.

**Figura 4**

*Creación del contenedor mongo-persistente*

```
$ podman run -d --name mongo-persistente \
    -p 27017:27017 \
    -v mongo-data:/data/db \
    -e MONGO_INITDB_ROOT_USERNAME=admin \
    -e MONGO_INITDB_ROOT_PASSWORD=Practica1 \
    docker.io/library/mongo:8
0eaeacc5dbae0d30b18fb45c67cc6f3c98d3d521f24d61de2d6add0209cdf7ac

$ podman ps
CONTAINER ID  IMAGE                       COMMAND     CREATED        STATUS        PORTS
NAMES
0eaeacc5dbae  docker.io/library/mongo:8  mongod      8 seconds ago  Up 8 seconds
0.0.0.0:27017->27017/tcp  mongo-persistente
```

*Nota.* La contraseña es únicamente de ejemplo para el entorno de la práctica.

### Paso 5. Inserción de datos con mongosh

Desde el anfitrión se ejecutó un script con `mongosh` que crea la base de datos `practica1`, inserta tres documentos de ejemplo en la colección `alumnos` y los consulta.

**Figura 5**

*Script insert.js*

```
db = db.getSiblingDB("practica1");
printjson(db.alumnos.insertMany([
  { numControl: "22020790", nombre: "Luis Daniel Morales Villarauz", carrera: "ISC",
semestre: 9 },
  { numControl: "22020001", nombre: "Ana López Pérez", carrera: "ISC", semestre: 9 },
  { numControl: "22020002", nombre: "Carlos Ruiz Díaz", carrera: "ISC", semestre: 9 }
]));
printjson(db.alumnos.find({}, { _id: 0 }).toArray());
print("Total de documentos: " + db.alumnos.countDocuments());
```

**Figura 6**

*Ejecución del script y documentos insertados*

```
$ mongosh "mongodb://admin:Practica1@localhost:27017/?authSource=admin" --quiet
insert.js
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6adef34f724b2e6c84521025'),
    '1': ObjectId('6adef34f724b2e6c84521026'),
    '2': ObjectId('6adef34f724b2e6c84521027')
  }
}
[
  {
    numControl: '22020790',
    nombre: 'Luis Daniel Morales Villarauz',
    carrera: 'ISC',
    semestre: 9
  },
  {
    numControl: '22020001',
    nombre: 'Ana López Pérez',
    carrera: 'ISC',
    semestre: 9
  },
  {
    numControl: '22020002',
    nombre: 'Carlos Ruiz Díaz',
    carrera: 'ISC',
    semestre: 9
  }
]
Total de documentos: 3
```

**Paso 6. Eliminación del contenedor**

Se detuvo y eliminó el contenedor. La lista de contenedores queda vacía, pero el volumen `mongo-data` sigue existiendo y conserva los archivos de datos de MongoDB (motor WiredTiger). Como el modo *rootless* asigna a los archivos un UID del espacio de nombres del usuario, se listaron con `podman unshare`.

**Figura 7**

*El contenedor se elimina y el volumen permanece*

```
$ podman stop mongo-persistente
mongo-persistente
$ podman rm mongo-persistente
mongo-persistente

$ podman ps -a --filter name=mongo
CONTAINER ID  IMAGE       COMMAND     CREATED     STATUS      PORTS       NAMES

$ podman volume ls --filter name=mongo-data
DRIVER      VOLUME NAME
local       mongo-data

$ podman unshare ls ~/.local/share/containers/storage/volumes/mongo-data/_data
collection-5b628eba-95e3-4d18-b214-fcfc87965556.wt
collection-9a703342-dc51-4a4d-b7c0-fedbe602eb35.wt
collection-aae08d8f-b714-45dd-bcd3-216ae2193f64.wt
collection-e73e1e0a-006d-4cb0-87de-a934ddab468a.wt
collection-ea30431e-2160-4af2-9f65-86a23009cfd4.wt
diagnostic.data
index-0bfd685b-845a-4f87-805f-af5abd8f582f.wt
index-2954cd19-27d2-4d03-8fbd-ffe7aa757471.wt
index-5e127e9d-dae3-4df3-bce7-0799f9a97df6.wt
index-6c5610ca-01de-4382-9b3b-38dffc97b8f6.wt
index-7750e4d0-455c-4913-b3f5-b5ea9cf19da2.wt
index-8054f41d-7f64-4c26-856d-5c0836c7f1bc.wt
index-ca8cae63-c817-42e2-b691-4b1c5ea917e3.wt
journal
_mdb_catalog.wt
mongod.lock
sizeStorer.wt
storage.bson
_tmp
WiredTiger
```

### Paso 7. Recreación del contenedor y verificación de la persistencia

Se creó un contenedor nuevo, `mongo-persistente-2`, montando el mismo volumen. Esta vez no se pasaron las variables de entorno del usuario administrador; aun así, la conexión con `admin` funciona, porque el usuario quedó guardado en el volumen junto con los datos. La consulta muestra que la base `practica1` y sus tres documentos siguen disponibles.

**Figura 8**

*Contenedor nuevo con el mismo volumen*

```
$ podman run -d --name mongo-persistente-2 \
    -p 27017:27017 \
    -v mongo-data:/data/db \
    docker.io/library/mongo:8
57dfe36b41dc8943d7fe11d44b1b1eeb169acc765fa5862f8b67f205466dbcf3

$ podman ps
CONTAINER ID  IMAGE                       COMMAND     CREATED        STATUS        PORTS
NAMES
57dfe36b41dc  docker.io/library/mongo:8  mongod      8 seconds ago  Up 8 seconds
0.0.0.0:27017->27017/tcp  mongo-persistente-2
```

**Figura 9**

*Los datos persisten en el contenedor nuevo*

```
$ cat verify.js
db = db.getSiblingDB("practica1");
printjson(db.getMongo().getDBNames());
printjson(db.alumnos.find({}, { _id: 0, numControl: 1, nombre: 1 }).toArray());
print("Total de documentos: " + db.alumnos.countDocuments());

$ mongosh "mongodb://admin:Practica1@localhost:27017/?authSource=admin" --quiet
verify.js
[
  'admin',
  'config',
  'local',
  'practica1'
]
[
  {
    numControl: '22020790',
    nombre: 'Luis Daniel Morales Villarauz'
  },
  {
    numControl: '22020001',
    nombre: 'Ana López Pérez'
  },
  {
    numControl: '22020002',
    nombre: 'Carlos Ruiz Díaz'
  }
]
Total de documentos: 3
```

### Limpieza del entorno

Para liberar los recursos al terminar, basta con eliminar el contenedor y, solo si ya no se necesitan los datos, el volumen. Hay que tener en cuenta que `podman volume rm` borra la información de forma definitiva.

**Figura 10**

*Comandos de limpieza*

```
$ podman rm -f mongo-persistente-2
$ podman volume rm mongo-data
```

## Conclusión

La práctica demostró que el ciclo de vida de los datos puede separarse del ciclo de vida del contenedor. Al montar un volumen con nombre en `/data/db`, MongoDB escribe sus archivos fuera de la capa efímera del contenedor, de modo que eliminarlo no implica perder la información: un contenedor nuevo que monte el mismo volumen recupera la base de datos completa, incluidos los usuarios y las colecciones.

Este esquema es la base para operar bases de datos NoSQL en contenedores, ya que permite actualizar la imagen, recrear el servicio o moverlo de configuración sin afectar los datos. Además, Podman permitió hacerlo todo en modo *rootless*, lo que reduce los riesgos de seguridad al no requerir privilegios de administrador.

## Referencias

Docker, Inc. (s.f.). *mongo: Docker official image*. Docker Hub. https://hub.docker.com/_/mongo

MongoDB, Inc. (s.f.). *MongoDB Shell (mongosh)*. MongoDB Documentation. https://www.mongodb.com/docs/mongodb-shell/

Podman Project. (s.f.-a). *podman-run: Run a command in a new container*. Podman Documentation. https://docs.podman.io/en/latest/markdown/podman-run.1.html

Podman Project. (s.f.-b). *podman-volume-create: Create a new volume*. Podman Documentation. https://docs.podman.io/en/latest/markdown/podman-volume-create.1.html